Snowpark Container Services(SPCS)

> [!IMPORTANT]
>
> - SPCS is available only on certain AWS regions and not available for trial accounts
> - All Snowpark Containers are run using a defined compute pool.

Compute Pool

List of available instance families[1]

- CPU_X64_XS
- CPU_X64_S
- CPU_X64_M
- CPU_X64_L
- HIGHMEM_X64_S
- HIGHMEM_X64_M
- HIGHMEM_X64_L
- GPU_NV_S
- GPU_NV_M
- GPU_NV_L

Create

Create a compute pool named `my_xs_compute_pool` with family `CPU_X64_XS`,

```
snow spcs compute-pool create my_xs_compute_pool \
  --family CPU_X64_XS
```

Create with `if not exists`,

```
snow spcs compute-pool create my_xs_compute_pool \
  --family CPU_X64_XS --if-not-exists
```

Create with initially suspended (**default**: not suspended initially),

```
snow spcs compute-pool create my_xs_compute_pool \
  --family CPU_X64_XS --init-suspend
```

Create with auto suspend(**default**: `3600 secs`) set to `2 mins(120 secs)` ,

```
snow spcs compute-pool create my_xs_compute_pool \
  --family CPU_X64_XS --auto-suspend-secs=120
```

Create with minimum nodes(scale down) as `1`(**default**) and maximum nodes(scale up) as `3`

```
snow spcs compute-pool create my_xs_compute_pool \
  --family CPU_X64_XS --min-nodes=1 --max-nodes=3
```

Create with auto resume on service/job request,

```
snow spcs compute-pool create my_xs_compute_pool \
  --family CPU_X64_XS --auto-resume
```

Create with auto-resume disabled,

> [!NOTE] Auto Resume disabled requires the compute pool to be started manually.

```
snow spcs compute-pool create my_xs_compute_pool \
  --family CPU_X64_XS --no-auto-resume
```

List Compute Pool

List all available compute pools for current role,

```
snow spcs compute-pool list
```

List compute pools like `my_xs%`

```
snow spcs compute-pool list --like 'my_xs%'
```

Describe Compute Pool

Get details about a compute pool,

```
snow spcs compute-pool describe my_xs_compute_pool
```

Status of Compute Pool

To know the current status of a compute pool,

```
snow spcs compute-pool status my_xs_compute_pool
```

Suspend a Compute Pool

Suspend a compute pool,

```
snow spcs compute-pool suspend my_xs_compute_pool
```

Resume a Compute Pool

Resume a compute pool,

```
snow spcs compute-pool resume my_xs_compute_pool
```

Properties on Compute Pool

You can `set/unset` the following properties on a compute pool after it's created,

Option	Description
`--min-nodes`	Minimum Node(s)
`--max-nodes`	Maximum Nodes(s)
`--auto-resume`	Enable Auto Resume
`--no-auto-resume`	Disable Auto Resume
`--auto-suspend-secs`	Auto Suspend in seconds
`--comment`	Comment

Set

Add a `comment` to the compute pool,

```
snow spcs compute-pool set --comment 'my small compute pool'
my_xs_compute_pool
```

Unset

Remove the `comment` from compute pool,

```
snow spcs compute-pool unset --comment my_xs_compute_pool
```

Delete all services on Compute Pool

Delete all services running on a compute pool

```
snow spcs compute-pool stop-all my_xs_compute_pool
```

Drop Compute Pool

Drop the compute pool,

```
snow spcs compute-pool drop my_xs_compute_pool
```

Image Registry

Login

> [!IMPORTANT] This requires Docker on local system

```
snow spcs image-registry login
```

Token

Get `current user` token to access image registry,

```
snow spcs image-registry token
```

Registry URL

Get image registry URL,

```
snow spcs image-registry url
```

Image Repository

> [!IMPORTANT]
>
> - A Database and Schema is required to create the Image Repository
> - Services can't be created using `ACCOUNTADMIN`, a custom role is required

The SQL script defines role, grants and warehouse.

As `ACCOUNTADMIN` run the script to setup required Snowflake resources,

- A Role named `cheatsheets_spcs_demo_role` to create Snowpark Container Services
- A Database named `CHEATSHEETS_DB` where the services will be attached to
- A Schema named `DATA_SCHEMA` on DB `CHEATSHEETS_DB` to hold the image repository.
- A Warehouse `cheatsheets_spcs_wh_s` which will be used to run query from services.

Set your Snowflake account user name,

```
export SNOWFLAKE_USER=<your snowflake user>
```

Run the `spcs_setup.sql` create the aforementioned Snowflake objects,

```
curl https://raw.githubusercontent.com/Snowflake-Labs/sf-
cheatsheets/main/samples/spcs_setup.sql |
snow sql --stdin
```

Create

Create a image repository named `my_image_repository`,

```
snow spcs image-repository create my_image_repository \
  --database='CHEATSHEETS_DB' \
  --schema='DATA_SCHEMA' \
  --role='cheatsheets_spcs_demo_role'
```

Create with if not exists,

```
snow spcs image-repository create my_image_repository \
  --database='CHEATSHEETS_DB' \
  --schema='DATA_SCHEMA' \
  --role='cheatsheets_spcs_demo_role' \
  --if-not-exists
```

Replace image repository `my_image_repository`,

```
snow spcs image-repository create my_image_repository \
  --database='CHEATSHEETS_DB' \
  --schema='DATA_SCHEMA' \
  --role='cheatsheets_spcs_demo_role' \
  --replace
```

List Image Repositories

List all image repositories in the database and schema,

```
snow spcs image-repository list \
  --database='CHEATSHEETS_DB' \
  --schema='DATA_SCHEMA' \
  --role='cheatsheets_spcs_demo_role'
```

URL

Get URL of the image repository `my_image_repository`,

```
snow spcs image-repository url my_image_repository \
  --database='CHEATSHEETS_DB' \
  --schema='DATA_SCHEMA' \
  --role='cheatsheets_spcs_demo_role'
```

List Images

Let us push a sample image to repository,

```
IMAGE_REPOSITORY=$(snow spcs image-repository url
my_image_repository \
  --database='CHEATSHEETS_DB' \
  --schema='DATA_SCHEMA'  \
  --role='cheatsheets_spcs_demo_role')
docker pull --platform=linux/amd64 nginx
docker tag nginx "$IMAGE_REPOSITORY/nginx"
docker push "$IMAGE_REPOSITORY/nginx"
```

List all images in repository `my_image_repository`,

```
snow spcs image-repository list-images my_image_repository \
  --database='CHEATSHEETS_DB' \
  --schema='DATA_SCHEMA' \
  --role='cheatsheets_spcs_demo_role'
```

List Image Tags

List all tags for image `nginx` in repository `my_image_repository`,

> [!IMPORTANT] The `--image-name` should be fully qualified name. Use `list-images` to get the fully qualified image name

```
snow spcs image-repository list-tags my_image_repository \
  --image-name=/CHEATSHEETS_DB/DATA_SCHEMA/my_image_repository/
nginx \
  --database='CHEATSHEETS_DB' \
  --schema='DATA_SCHEMA' \
  --role='cheatsheets_spcs_demo_role'
```

Drop

```
snow spcs image-repository drop my_image_repository \
  --database='CHEATSHEETS_DB' \
  --schema='DATA_SCHEMA' \
  --role='cheatsheets_spcs_demo_role'
```

Services

Create a SPCS service specification[2] file,

> [!TIP] Tools like jq can help extract data from the command output e.g. to get the image name
>
> ```
> export IMAGE=$(snow spcs image-repository list-images
> my_image_repository \
> ```

```
    --database='CHEATSHEETS_DB' \
    --schema='DATA_SCHEMA' --format json | jq -r '.[0].image')
```

```
cat <<EOF | tee work/service-spec.yaml
spec:
  containers:
    - name: nginx
      image: $IMAGE
      readinessProbe:
        port: 80
        path: /
  endpoints:
    - name: nginx
      port: 80
      public: true
EOF
```

Create a Service named `nginx` using compute pool `my_xs_compute_pool` and specification `work/service-spec.yaml`,

```
snow spcs service create nginx \
  --compute-pool=my_xs_compute_pool \
  --spec-path=work/service-spec.yaml \
  --database='CHEATSHEETS_DB' \
  --schema='DATA_SCHEMA' \
  --role='cheatsheets_spcs_demo_role'
```

Create a Service if not exists,

```
snow spcs service create nginx \
  --compute-pool=my_xs_compute_pool \
  --spec-path=work/service-spec.yaml \
  --if-not-exists \
  --database='CHEATSHEETS_DB' \
  --schema='DATA_SCHEMA' \
  --role='cheatsheets_spcs_demo_role'
```

Create with minimum instances `1` (**default**) and maximum instances to be `3`,

```
snow spcs service create nginx \
  --compute-pool=my_xs_compute_pool \
  --spec-path=work/service-spec.yaml \
  --min-instances=1 \
  --max-instances=3 \
  --database='CHEATSHEETS_DB' \
  --schema='DATA_SCHEMA' \
  --role='cheatsheets_spcs_demo_role'
```

Create service that uses a specific warehouse `cheatsheets_spcs_wh_s` for all its queries,

```
snow spcs service create nginx \
  --compute-pool=my_xs_compute_pool \
  --spec-path=work/service-spec.yaml \
  --query-warehouse='cheatsheets_spcs_wh_s' \
  --database='CHEATSHEETS_DB' \
  --schema='DATA_SCHEMA' \
  --role='cheatsheets_spcs_demo_role'
```

Status

Check service status,

> [!NOTE] It will take few minutes for the service to be in `READY` status

```
snow spcs service status nginx \
  --database='CHEATSHEETS_DB' \
  --schema='DATA_SCHEMA' \
  --role='cheatsheets_spcs_demo_role'
```

Describe

Get more details about the service,

```
snow spcs service describe nginx \
  --database='CHEATSHEETS_DB' \
  --schema='DATA_SCHEMA' \
  --role='cheatsheets_spcs_demo_role'
```

Check Logs of Service

Check the logs of service with the container named `nginx` with instance `0`,

> [!NOTE] Find `instanceId` and `containerName` using the command `describe` command.

```
snow spcs service logs nginx \
  --container-name=nginx \
  --instance-id=0 \
  --database='CHEATSHEETS_DB' \
  --schema='DATA_SCHEMA' \
  --role='cheatsheets_spcs_demo_role'
```

List

List all available services,

```
snow spcs service list  \
  --database='CHEATSHEETS_DB' \
```

```
  --schema='DATA_SCHEMA' \
  --role='cheatsheets_spcs_demo_role'
```

Query services `in` database,

```
snow spcs service list  --in database CHEATSHEETS_DB \
  --role='cheatsheets_spcs_demo_role'
```

Query services `in` database and like `ng%`,

```
snow spcs service list  --in database CHEATSHEETS_DB --like 'ng%'
\
  --role='cheatsheets_spcs_demo_role'
```

Service Endpoints

List the service endpoint for the service `nginx`,

```
snow spcs service list-endpoints nginx  \
  --database='CHEATSHEETS_DB' \
  --schema='DATA_SCHEMA' \
  --role='cheatsheets_spcs_demo_role'
```

> [!NOTE] Open the `ingress_url` on the browser will take you to NGINX home page after authentication

Suspend a service

Suspend the service,

```
snow spcs service suspend nginx  \
  --database='CHEATSHEETS_DB' \
  --schema='DATA_SCHEMA' \
  --role='cheatsheets_spcs_demo_role'
```

Resume a service

Resume the service,

```
snow spcs service resume nginx  \
  --database='CHEATSHEETS_DB' \
  --schema='DATA_SCHEMA' \
  --role='cheatsheets_spcs_demo_role'
```

> [!NOTE] Resume service will take few minutes, use the `status` command to check the status

Supported properties on Service

You can `set/unset` the following properties on a service even after it's created,

Option	Description
`--min-instances`	Minimum number of service instance(s), typically used while scaling down
`--max-instances`	Maximum number of service instance(s), typically used while scaling up
`--auto-resume`	Enable auto resume
`--no-auto-resume`	Disable auto resume
`--query-warehouse`	The Warehouse to use while doing query from the service
`--comment`	Comment for the service

Set

Add a comment to the service,

```
snow spcs service set --comment 'the nginx service' nginx  \
  --database='CHEATSHEETS_DB' \
  --schema='DATA_SCHEMA' \
  --role='cheatsheets_spcs_demo_role'
```

Use service `describe` to check on the updated property

Unset

Remove the comment from the service,

```
snow spcs service unset --comment nginx  \
  --database='CHEATSHEETS_DB' \
  --schema='DATA_SCHEMA' \
  --role='cheatsheets_spcs_demo_role'
```

Upgrade

Upgrade the service `nginx` with new specification e.g a tag upgrade or probe updates etc.,

```
snow spcs service upgrade nginx \
  --spec-path=work/service-spec_V2.yaml \
  --database='CHEATSHEETS_DB' \
  --schema='DATA_SCHEMA' \
  --role='cheatsheets_spcs_demo_role'
```

Drop

Drop a service named `nginx`

```
snow spcs service drop nginx \
  --database='CHEATSHEETS_DB' \
  --schema='DATA_SCHEMA' \
  --role='cheatsheets_spcs_demo_role'
```

> [!NOTE] SPCS has compute associated with it, run the clean up script to clean the Snowflake resources created as part of this cheatsheet.
>
> ```
> curl https://raw.githubusercontent.com/Snowflake-Labs/sf-cheatsheets/main/samples/spcs_cleanup.sql |
> snow sql --stdin
> ```

Quickstarts

- Snowflake Developers::Quickstart
- Snowflake Developers::Getting Started With Snowflake CLI
- Intro to Snowpark Container Services
- Build a Data App and run it on Snowpark Container Services

Documentation

- Snowflake CLI
- Execute Immediate Jinja Templating
- Snowpark Container Services

Tutorials

- Snowpark Container Services Tutorial

---

1. https://docs.snowflake.com/en/sql-reference/sql/create-compute-pool↩

2. https://docs.snowflake.com/en/developer-guide/snowpark-container-services/specification-reference↩